



3강. 리눅스와 로봇 개발환경



학습 목표

- 로봇 개발에서 리눅스(Ubuntu)가 표준이 된 이유를 설명할 수 있습니다.
- 리눅스의 핵심 개념 — 커널과 사용자 공간, 프로세스, 파일시스템 계층, "모든 것은 파일" 철학 — 을 설명할 수 있습니다.
- 파일 권한(rwx)과 소유자 모델을 이해하고, `sudo` 가 필요한 상황과 위험을 판단할 수 있습니다.
- 셸(bash)에서 파일·프로세스를 다루는 필수 명령을 사용할 수 있습니다.
- SSH로 로봇 온보드 컴퓨터에 원격 접속하고, udev rules로 장치 경로를 고정할 수 있습니다.



목차

들어가며

1. 왜 로봇은 리눅스를 쓸까요?

2. 리눅스의 구조 — 커널과 사용자 공간

프로세스 (Process)

3. 파일시스템 — "모든 것은 파일"

경로 표기

4. 사용자와 권한 — rwx와 sudo

로봇 개발에서 권한이 문제 되는 순간

sudo — 관리자 권한의 양날

5. 셸(bash) — 필수 명령 모음

6. SSH — 모니터 없는 로봇에 접속하기

7. udev rules — 장치 경로 고정하기

미니 퀴즈

실습

정리

들어가며

로봇을 개발하러 가면 가장 먼저 마주치는 것은 로봇이 아니라 **검은 터미널 화면**입니다. 로봇의 온보드 컴퓨터에는 보통 모니터도 키보드도 없습니다. 우리는 노트북에서 원격으로 접속해, 명령어로 프로그램을 실행하고, 로그를 확인하고, 센서가 연결됐는지 점검합니다.

그 세계의 운영체제가 **리눅스(Linux)**, 그중에서도 **Ubuntu**입니다. ROS2의 공식 지원 플랫폼이 Ubuntu이고, Jetson 같은 Edge 컴퓨터도 Ubuntu 기반으로 돌아갑니다. 즉 리눅스는 선택이 아니라 **로봇 개발의 공용어**입니다. 이번 강의에서는 이 공용어의 문법 — 커널·프로세스·파일시스템·권한·셸 — 을 익히고, 로봇 현장에서 반드시 만나는 SSH와 udev까지 다루겠습니다.

1. 왜 로봇은 리눅스를 쓸까요?

Windows나 macOS 대신 리눅스가 로봇의 표준이 된 데는 분명한 이유들이 있습니다.

- **오픈소스·무료**: 로봇 100대를 만들어도 OS 라이선스 비용이 없습니다. 그리고 커널 소스가 공개돼 있어 **필요하면 OS 수준까지 손댈 수 있습니다**(예: 실시간 패치 PREEMPT_RT — 2강 참조).
- **가볍고 조립 가능**: GUI 없이 필요한 부품만 얹어 실행할 수 있어, 자원이 제한된 온보드 컴퓨터에 알맞습니다.
- **강력한 원격·자동화 도구**: SSH, 셸 스크립트, 서비스 자동 실행(systemd) 등 "모니터 없는 컴퓨터"를 다루는 도구가 처음부터 갖춰져 있습니다.
- **생태계**: ROS2, OpenCV, PyTorch 등 로봇·AI 소프트웨어의 1차 지원 대상이 리눅스입니다. 문제가 생겼을 때 찾을 수 있는 자료도 압도적으로 많습니다.

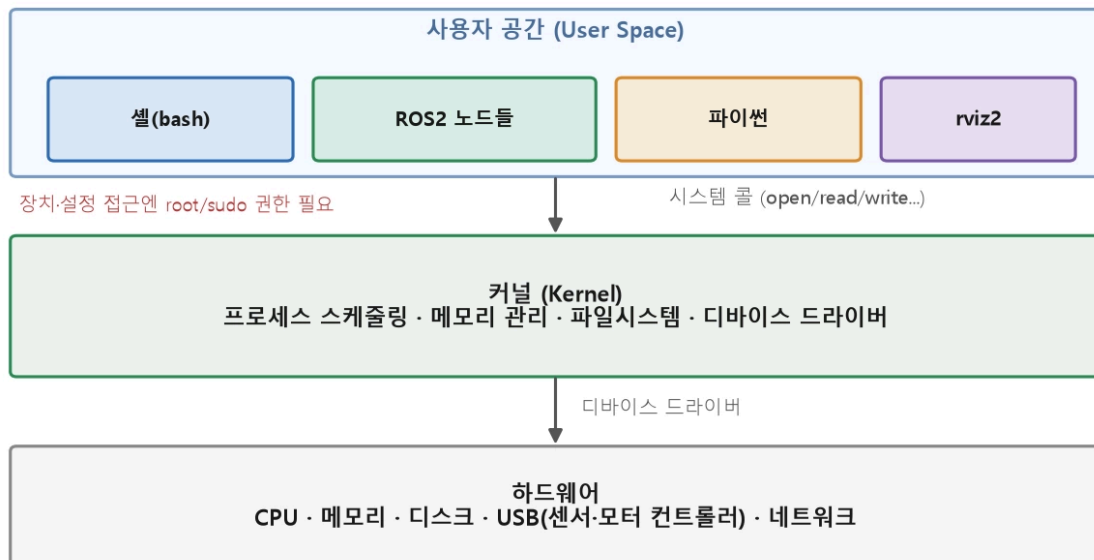
✔ **핵심**: 로봇 개발자에게 리눅스는 "여러 OS 중 하나"가 아니라, ROS2·Edge 컴퓨터·서버를 관통하는 **기본 환경**입니다.

2. 리눅스의 구조 — 커널과 사용자 공간

리눅스 시스템은 크게 두 세계로 나뉩니다. 우리가 짜는 프로그램이 어떻게 하드웨어(센서·모터)까지 닿는지 이 구조로 이해할 수 있습니다.

사용자 공간 - 커널 - 하드웨어: 프로그램이 장치에 닿는 길

일반 사용자 권한으로 실행



images/03_커널_계층.png — 사용자 공간 - 커널 - 하드웨어

💡 **해석:** 우리가 실행하는 모든 프로그램(셸, ROS2 노드, 파이썬, RViz)은 **사용자 공간**에서 돌립니다. 프로그램이 파일을 읽거나 USB 장치에 접근하려면 반드시 **커널**에 **시스템 콜**(open/read/write 등)로 요청해야 하고, 커널이 **디바이스 드라이버**를 통해 실제 하드웨어를 조작합니다. 프로그램이 하드웨어를 직접 만질 수 없게 한 이 구조 덕분에, 프로그램 하나가 죽어도 시스템 전체는 살아남습니다.

프로세스 (Process)

실행 중인 프로그램 하나하나를 **프로세스**라고 합니다. 각 프로세스는:

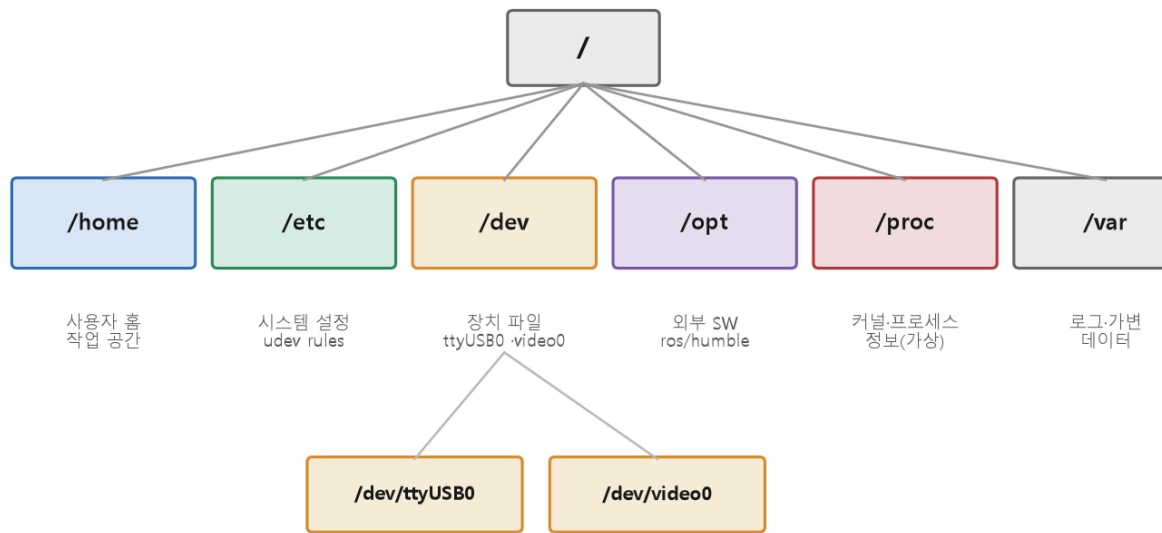
- 고유 번호 **PID**(Process ID)를 가지며,
- **독립된 메모리 공간**을 배정받아 서로 침범할 수 없고,
- 커널 스케줄러가 CPU 시간을 배분합니다 (2강의 지터가 여기서 발생했지요).

로봇을 켜면 수십 개의 ROS2 노드가 각각 프로세스로 뜹니다. `ps aux` 나 `htop` 으로 확인하고, 멈춘 노드는 `kill PID` 로 종료합니다. "노드 하나가 죽어도 다른 노드는 계속 돈다"는 ROS2의 견고함은 바로 이 **프로세스 격리** 위에 서 있습니다.

3. 파일시스템 — "모든 것은 파일"

리눅스 파일시스템은 Windows의 `C:\`, `D:\` 같은 드라이브 문자가 없고, **루트(/)** 하나에서 시작하는 단일 트리입니다.

리눅스 파일시스템 계층 (로봇 개발 관점)



센서·모터 컨트롤러가 이런 '장치 파일'로 나타납니다 — 모든 것이 파일!

images/03_파일시스템_트리.png — 리눅스 파일시스템 계층

💡 **해석:** 로봇 개발에서 자주 만나는 디렉터리들입니다. `/home` 은 내 작업 공간, `/etc` 는 시스템 설정(udev rules 포함), `/opt` 는 ROS2 같은 외부 소프트웨어, `/proc` 은 커널이 만들어 주는 가상 정보 창구입니다. 가장 특별한 곳은 `/dev` — 하드웨어 장치가 **파일의 모습으로** 나타나는 곳입니다.

리눅스의 유명한 설계 철학이 "**모든 것은 파일(everything is a file)**" 입니다. 키보드도, USB 시리얼 포트도, 카메라도 전부 파일처럼 열고(open) 읽고(read) 쓸(write) 수 있습니다.

- USB로 연결한 모터 컨트롤러 → `/dev/ttyUSB0` (시리얼 장치 파일)
- USB 카메라 → `/dev/video0`
- 첫 번째 디스크 → `/dev/sda`

이 철학이 강력한 이유는, **하드웨어를 다루는 코드가 파일을 다루는 코드와 같아진다는** 데 있습니다. 파이썬에서 `open('/dev/ttyUSB0')` 로 모터 컨트롤러와 대화할 수 있는 것이지요.

경로 표기

- **절대 경로:** 루트부터 전부 적기 — `/home/robot/ws/src/my_node.py`
- **상대 경로:** 현재 위치 기준 — `src/my_node.py`, `..` (한 단계 위), `.` (현재), `~` (내 홈 디렉터리)

4. 사용자와 권한 — rwx와 sudo

리눅스는 태생부터 **다중 사용자 시스템**입니다. 모든 파일에는 소유자가 있고, 세 부류(소유자 u / 그룹 g / 그 외 o)에 대해 **읽기 r / 쓰기 w / 실행 x** 권한이 각각 지정됩니다.

`ls -l` 출력의 앞부분을 읽는 법:

```
-rwxr-xr-- 1 robot dialout 5820 Jul 7 10:30 motor_test.py
```

조각	의미
-	종류 (- 일반 파일, d 디렉터리, c 문자 장치)
rwx	소유자(robot): 읽기+쓰기+실행 가능
r-x	그룹(dialout): 읽기+실행만 가능
r--	그 외: 읽기만 가능

권한 변경은 `chmod` (권한), `chown` (소유자)으로 합니다. 예: `chmod +x run.sh` (실행 권한 부여).

로봇 개발에서 권한이 문제 되는 순간

로봇에서 가장 흔한 권한 이슈는 **시리얼 포트 접근**입니다. `/dev/ttyUSB0` 은 보통 `dialout` 그룹 소유라서, 내 계정이 그 그룹에 없으면 `Permission denied` 가 납니다. 해결은:

```
sudo usermod -aG dialout $USER # 내 계정을 dialout 그룹에 추가 # 로그아웃 후 재로그인해야 적용됩니다
```

sudo — 관리자 권한의 양날

`sudo` 는 명령 하나를 **root(관리자) 권한**으로 실행합니다. 시스템 설정 변경(`/etc` 수정), 패키지 설치(`apt install`), 서비스 제어에 필요합니다. 그러나:

⚠ **주의:** `sudo` 는 시스템의 모든 것을 지울 수 있는 권한입니다. 특히 "**에러가 나서 그냥 sudo를 붙였다**"는 습관은 금물입니다. 권한 에러의 원인(그룹 미소속, 소유자 문제)을 파악해 바로잡는 것이 정석이고, `sudo` 는 정말 시스템 수준 변경이 필요할 때만 씁니다. 예를 들어 ROS2 워크스페이스를 `sudo colcon build` 하면 빌드 산출물이 root 소유가 되어 이후 일반 권한 빌드가 계속 깨지는, 대표적인 자충수가 됩니다.

5. 셸(bash) — 필수 명령 모음

셸(shell) 은 명령어를 해석해 커널에 전달하는 프로그램이고, Ubuntu의 기본 셸이 **bash** 입니다. 로봇 개발에서 매일 쓰는 명령을 범주별로 정리합니다.

범주	명령	용도
탐색	<code>pwd</code> / <code>ls -l</code> / <code>cd</code>	현재 위치 / 목록(권한 포함) / 이동
파일	<code>cp</code> / <code>mv</code> / <code>rm -r</code> / <code>mkdir -p</code>	복사 / 이동·이름변경 / 삭제 / 디렉터리 생성
내용	<code>cat</code> / <code>less</code> / <code>head</code> / <code>tail -f</code>	출력 / 페이지 보기 / 앞·뒤(-f는 실시간 로그 추적)
검색	<code>grep -r "패턴" 경로</code> / <code>find . -name "*.py"</code>	내용 검색 / 파일 검색
프로세스	<code>ps aux</code> / <code>htop</code> / <code>kill</code> / <code>kill -9</code>	목록 / 모니터 / 종료 요청 / 강제 종료
장치·시스템	<code>lsusb</code> / <code>dmesg tail</code> / <code>df -h</code>	USB 장치 확인 / 커널 메시지 / 디스크 용량
조합	<code> </code> (파이프) / <code>></code> (리다이렉트) / <code>&&</code>	명령 연결 / 출력 저장 / 성공 시 이어서

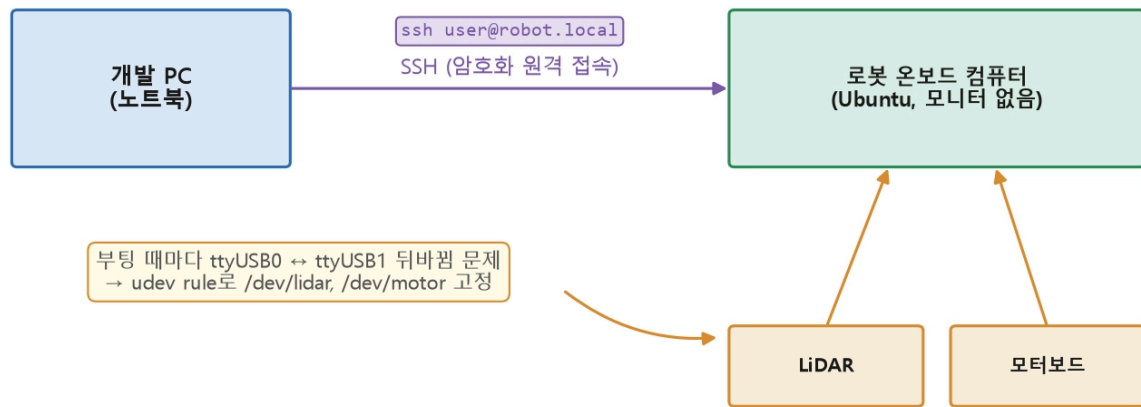
특히 **파이프(|)** 는 리눅스 철학("작은 도구를 조합하라")의 정수입니다:

```
# 실행 중인 ROS2 관련 프로세스만 골라 보기 ps aux | grep ros # 커널 메시지에서 USB 연결 이벤트만 실시간 추적 dmesg -w | grep -i usb
```

▶ **심화** — 환경 변수와 source, 그리고 ROS2의 setup.bash

6. SSH — 모니터 없는 로봇에 접속하기

로봇 온보드 컴퓨터는 **헤드리스(headless, 모니터 없음)** 로 운용됩니다. 여기에 접속하는 표준 수단이 **SSH(Secure Shell)** — 네트워크를 통한 **암호화된 원격 터미널**입니다.



images/03_SSH_udev.png — SSH 원격 접속과 udev 장치 고정

💡 **해석:** (위) 개발 PC에서 `ssh user@robot.local` 로 로봇에 접속하면, 이후 입력하는 모든 명령은 **로봇 위에서** 실행됩니다. (아래) 로봇에 LiDAR와 모터보드를 USB로 꽂으면 `/dev/ttyUSB0`, `ttyUSB1` 이 되는데, **부팅 순서에 따라 번호가 뒤바뀌는 문제**가 있습니다. udev rule로 `/dev/lidar`, `/dev/motor` 같은 고정 이름을 만들어 해결합니다.

기본 사용법:

```
ssh robot@192.168.0.42 # IP로 접속 ssh robot@myrobot.local # mDNS 호스트명으로 접속
scp data.bag robot@192.168.0.42:~/logs/ # 파일 복사 (SSH 기반)
```

매번 비밀번호를 치지 않으려면 **키 기반 인증**을 씁니다:

```
ssh-keygen -t ed25519 # 내 PC에서 키 쌍 생성 (1회) ssh-copy-id robot@192.168.0.42 # 공개키를 로봇에 등록
ssh robot@192.168.0.42 # 이후 비밀번호 없이 접속
```

원리는 비대칭 암호화입니다. 내 PC에 **개인키**, 로봇에 **공개키**를 두면, 접속 시 로봇이 "공개키로 잠금 문제"를 내고 개인키를 가진 내 PC만 풀 수 있습니다. 비밀번호가 네트워크로 전송되지 않으므로 더 안전하고, 스크립트 자동화에도 필수입니다.

7. udev rules — 장치 경로 고정하기

문제 상황을 구체적으로 봅시다. 로봇에 USB 장치가 두 개(LiDAR, 모터보드) 꽂혀 있습니다.

- 오늘 부팅: LiDAR = `/dev/ttyUSB0`, 모터보드 = `/dev/ttyUSB1`
- 내일 부팅: LiDAR = `/dev/ttyUSB1`, 모터보드 = `/dev/ttyUSB0` ← 순서가 바뀐!

코드에 `/dev/ttyUSB0` 을 하드코딩했다면, 내일은 모터 명령이 LiDAR로 날아갑니다. 해결책이 **udev rule** — "이런 특징을 가진 장치가 연결되면 이런 이름의 링크를 만들어라"라는 커널 장치 관리자(udev)의 규칙입니다.

절차는 세 단계입니다.

```
# 1) 장치의 고유 식별자(제조사 ID, 제품 ID, 시리얼) 알아내기 udevadm info -a -n /dev/ttyUSB0 | grep -E "idVendor|idProduct|serial" | head -3
# 2) /etc/udev/rules.d/99-robot.rules 작성 (sudo 필요) # idVendor 10c4, idProduct ea60인 장치를 /dev/lidar로 링크 SUBSYSTEM=="tty", ATTRS{idVendor}=="10c4", ATTRS{idProduct}=="ea60", SYMLINK+="lidar", MODE="0666" # 3) 규칙 재적용 sudo udevadm control --reload-rules && sudo udevadm trigger
```

이후 코드에서는 항상 `/dev/lidar` 를 열면 됩니다. 같은 모델 장치가 여러 개면 `ATTRS{serial}` (시리얼 번호)로 구분합니다. `MODE="0666"` 은 권한 문제(4장)까지 함께 해결해 주는 옵션입니다.

- ✔ **핵심:** 실제 로봇 배포에서 udev rule은 선택이 아니라 필수입니다. "재부팅했더니 로봇이 이상해졌다"는 신고의 단골 원인이 장치 번호 뒤바뀜입니다.

미니 퀴즈

💡 아래 문항을 먼저 스스로 풀어 본 뒤 실습으로 넘어가세요. 답안은 실습 결과물과 함께 제출합니다. (총 10점)

1. (2점) `ls -l` 결과가 `crw-rw---- 1 root dialout ... /dev/ttyUSB0` 일 때, 일반 사용자 robot 이 이 장치를 읽고 쓰려면 가장 적절한 조치는?

- ① `sudo chmod 777 /dev/ttyUSB0` 를 매번 실행한다
- ② robot 을 dialout 그룹에 추가한다
- ③ 매번 sudo 로 프로그램을 실행한다
- ④ 파일 소유자를 robot 으로 바꾼다

2. (2점) 리눅스에서 "모든 것은 파일"이라는 원칙이 로봇 개발에서 구체적으로 무엇을 뜻하는지, 예를 하나 들어 설명하세요.
3. (2점) `udev` 규칙을 쓰는 이유를 한 문장으로 설명하세요.
4. (2점) 실행 중인 프로세스를 정상 종료 요청할 때와 강제 종료할 때 쓰는 명령을 각각 쓰고, **강제 종료를 마지막 수단으로 쓰는 이유**를 한 줄로 쓰세요.
5. (2점) 참/거짓: `sudo` 는 권한 문제를 해결하는 가장 좋은 기본 습관이다.

실습

Ubuntu 환경(또는 WSL2)에서 다음을 수행해 보겠습니다.

1. **파일시스템 탐험:** `cd /` 후 `ls -l` 로 최상위 디렉터리를 살펴보고, `/dev` 에서 `ls -l /dev/tty*` 로 시리얼 장치 파일들의 종류 문자(`c`)와 그룹을 확인해 보세요.
2. **권한 실험:** 홈 디렉터리에 `test.sh` 를 만들고(`echo 'echo hello' > test.sh`), 실행을 시도해 보세요(`./test.sh`). 실패 이유를 `ls -l` 로 확인하고 `chmod +x` 로 해결해 보세요.
3. **프로세스 추적:** 터미널 하나에서 `sleep 300` 을 실행하고, 다른 터미널에서 `ps aux | grep sleep` 으로 PID를 찾아 `kill` 로 종료해 보세요.
4. **(심화) SSH 키 인증:** 가상머신·클라우드·라즈베리파이 등 접속 가능한 원격 리눅스가 있다면, 키 쌍을 만들어 비밀번호 없는 접속을 구성해 보세요.
5. **udev rule로 장치 경로 고정:** USB 시리얼 장치를 꽂고 `udevadm info -a -n /dev/ttyUSB0` 로 `idVendor` · `idProduct` · `serial` 을 확인한 뒤, `/etc/udev/rules.d/99-robot.rules` 를 작성해 `/dev/robot_lidar` 같은 고정 이름을 부여해 보세요. `sudo udevadm control --reload-rules && sudo udevadm trigger` 로 재적용하고, 장치를 뽑았다 다시 꽂아도 같은 이름이 유지되는지 확인합니다. 장치가 두 개라면 `ATTRS{serial}` 로 구분해 각각 다른 이름을 주세요.

각 단계에서 **무엇이 왜 그렇게 동작했는지** 한 줄씩 설명을 적는 것까지가 실습입니다.

!! 실습 5번은 **udev 데몬이 동작하는 환경**이 필요합니다. WSL2는 `udev`가 기본적으로 꺼져 있어 이 항목만 동작하지 않으니, 네이티브 설치나 가상머신에서 진행하세요. 1~4번은 WSL2에서도 그대로 됩니다.

정리

- 로봇 개발의 표준 OS는 **리눅스(Ubuntu)** — 오픈소스, 경량, 원격·자동화 도구, ROS2 생태계가 이유입니다.
- 프로그램(사용자 공간)은 **시스템 콜** → **커널** → **드라이버** 경로로만 하드웨어에 닿으